PUCP

PROGRAMACIÓN 3 (1INF30)



JDBC
In A Nutshell

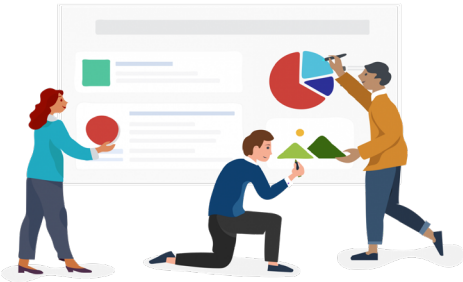
2026

Profesores:

- Dr. Freddy Paz
- Dr. José Luis Corcuera
- Mag. Eric Huiza
- Mag. David Allasi

AGENDA

1. Introducción a JDBC
2. La clase DriverManager e interfaz Connection
3. La interfaz java.sql.Statement
4. La interfaz java.sql.ResulSet
5. La interfaz java.sql.PreparedStatement
6. Manejo de transacciones
7. Buenas prácticas en el uso de JDBC
8. Modelamiento de Herencia en BD
9. El patrón Data Access Object (DAO)
10. Definición de módulos en el proyecto SoftPro



Introducción a JDBC

Introducción a JDBC

El propósito principal de JDBC es permitir que una aplicación se conecte y acceda a una fuente de datos, la cual puede ser un sistema de Gestión de Bases de Datos (DBMS), un sistema de archivos heredado (legacy file system), o cualquier otra fuente de información que cuente con su correspondiente controlador **JDBC**.

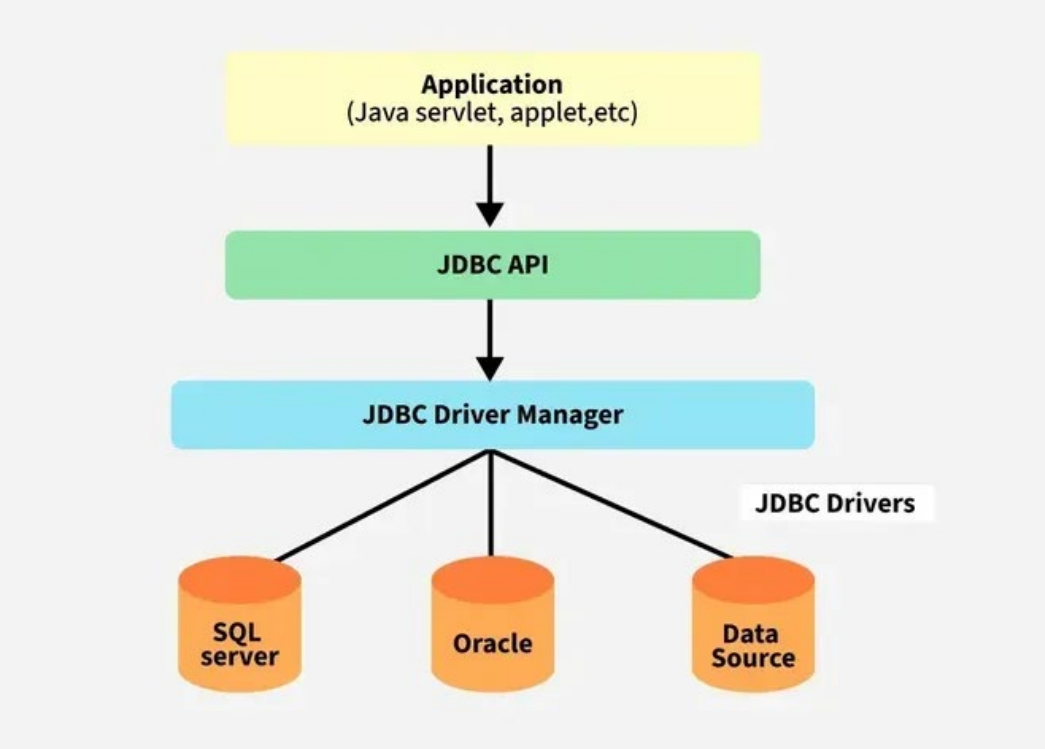


Image tomada de: https://www.geeksforgeeks.org/java/introduction-to-jdbc/

Introducción a JDBC

JDBC funciona como un intermediario que requiere un **controlador específico del fabricante** para comunicarse con la base de datos elegida (por ejemplo, com.mysql.cj.jdbc.Driver para MySQL o org.apache.derby.jdbc.EmbeddedDriver para Java DB).

Cada controlador contiene clases que implementan la interfaz java.sql.Driver entre otras interfaces definidas en el paquete java.sql.* (Connection, Statement, PreparedStatement, ResultSet, etc).

La clase DriverManager interfaz Connection

La clase DriverManager e interfaz Connection

- La clase **DriverManager** es una clase de JDBC que permite conectar una aplicación a una fuente de datos mediante una JDBC URL. Esta conección se logra a traves de la implementación de la interfaz java.sql.Connection dada por el **controlador JDBC**.

- Al intentar establecer una conexión por primera vez, esta clase carga automáticamente cualquier controlador JDBC 4.0 que se encuentre en la ruta de clases (class path).

- Para saber si el driver es version 4.0 dentro del driver JDBC (file *.jar) deberá existir el archivo META-INF/services/java.sql.Driver. (En caso no existir el archivo deberá cargar la clase del Driver usando Class.forName.

La clase DriverManager e interfaz Connection

El método DriverManager.getConnection requiere de una JDBC URL la cual permite especificar información del tipo de fuente de datos al cual deseamos conectarnos, dirección y puerto de este; y parametros adicionales que podemos especificar. El nombre de usuario y la contraseña necesarios para acceder al DBMS se pueden definir utilizando un objeto Properties.

Por ejemplo, una URL típica de MySQL para acceder a una base llamada "my-database" ubicada en el servidor my-server.com es:

jdbc:**mysql**://my-server.com:3306/my-database?param1=value1&...&paramN=valueN

La clase DriverManager e interfaz Connection

Ejemplo de obtención de conección:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.Properties;

// 1. Se crea un objeto Properties con las credenciales, tal como indican las fuentes
Properties credentials = new Properties();
credentials.put("user", "tu_usuario");
credentials.put("password", "tu_contraseña");

// 2. Se define la URL de la base de datos MySQL
String url = "jdbc:mysql://localhost:3306/my-database";

// 3. Se pasa la URL y las credenciales al método getConnection
Connection connection = DriverManager.getConnection(url, credentials);

// do your stuff here
connection.close();
```

La interfaz java.sql.Statement

La interfaz java.sql.Statement

- Un objeto de tipo **Statement** se utiliza para ejecutar comandos y consultas en la base de datos, lo que habitualmente genera un objeto **ResultSet** (una tabla que representa los resultados de la búsqueda).
- Interfaces más avanzadas como PreparedStatement y CallableStatement también extienden la interfaz Statement.
- La interfaz permite procesar comandos únicos mediante métodos como executeUpdate, el cual envía la instrucción a la base de datos y retorna un número que indica la cantidad de filas que fueron afectadas.

La interfaz java.sql.Statement

```
import java.sql.Statement;

// here I've already obtain a connection object

Statement stmt = connection.createStatement();

// Se envía un único comando UPDATE usando executeUpdate, también
puede enviar sentencias INSERT o DELETE
String sql = "UPDATE COFFEES SET PRICE = 9.99 WHERE COF_NAME =
'Amaretto'";
int filasAfectadas = stmt.executeUpdate(sql);

stmt.close();
connection.close();
```

La interfaz java.sql.Statement

- En el ejemplo anterior de código que pasaría si el valor de COF_NAME es recibido en una variable llamada cofName entonces nuestra sentencia SQL cambiaría a:

```
String sql = "UPDATE COFFEES SET PRICE = 9.99 WHERE COF_NAME = '" + cofName + "'";
```

- ¿Qué problema puede generar la sentencia SQL anterior?

SQL INJECTION

La interfaz java.sql.ResultSet

La interfaz java.sql.ResultSet

- Es la representación en forma de tabla de los datos obtenidos tras ejecutar una consulta SQL en la base de datos.
- Emplea un "cursor" o puntero para recorrer la información fila por fila. Este cursor inicia su recorrido justo antes del primer registro y avanza utilizando el método next(), el cual te indica (true o false) si todavía existen registros disponibles por leer.
- Para recuperar los datos se hace uso de métodos getter (como getString o getInt, etc) para extraer los valores precisos de la fila en la que se encuentra posicionado el cursor. Se puede ubicar la columna que deseas leer usando su número de índice (comenzando siempre desde el 1) o proporcionando su nombre o alias.

La interfaz java.sql.ResultSet

```
import java.sql.Connection;
import java.sql.Statement;
import java.sql.ResultSet;

// here I've already obtain a connection object
Statement stmt = con.createStatement();

// Se genera el ResultSet al ejecutar una consulta SELECT
ResultSet rs = stmt.executeQuery("SELECT COF_NAME, SUPPLIER_ID FROM COFFEES");

// rs.next() mueve el cursor a la siguiente fila. Devuelve false si no hay más filas.
while (rs.next()) {
    // Se recuperan los valores usando el nombre de la columna
    String nombreCafe = rs.getString("COF_NAME");
    int idProveedor = rs.getInt("SUPPLIER_ID");

    System.out.println("Café: " + nombreCafe + ", Proveedor: " + idProveedor);
}

rs.close();
stmt.close();
connection.close();
```

La interfaz java.sql.ResultSet

```
import java.sql.Connection;
import java.sql.Statement;
import java.sql.ResultSet;

// here I've already obtain a connection object
Statement stmt = con.createStatement();

// Se genera el ResultSet al ejecutar una consulta SELECT
ResultSet rs = stmt.executeQuery("SELECT COF_NAME, SUPPLIER_ID FROM COFFEES");

// rs.next() avanza el cursor fila por fila
while (rs.next()) {
    // Se recuperan los valores usando el NÚMERO de la columna en lugar del nombre
    String nombreCafe = rs.getString(1); // 1 = COF_NAME
    int idProveedor = rs.getInt(2);      // 2 = SUPPLIER_ID

    System.out.println("Café: " + nombreCafe + ", Proveedor: " + idProveedor);
}

rs.close();
stmt.close();
connection.close();
```

La interfaz java.sql.PreparedStatement

La interfaz java.sql.PreparedStatement

- Es un objeto derivado de Statement para enviar sentencias SQL (INSERT, UPDATE, DELETE, SELECT, etc) a la base de datos.
- Ofrece alto rendimiento dado que la consulta se precompila al crearse, lo que permite ejecuciones repetidas de forma mucho más rápida.
- Permite el uso de parámetros dinámicos (?). Utiliza signos de interrogación como marcadores en lugar de concatenar texto. Estos se inyectan usando métodos setter (ej. setString)
- Ofrecen máxima seguridad. Evita la inyección SQL al tratar los datos del usuario estrictamente como valores y nunca como código ejecutable.
- Expone métodos como executeQuery o executeUpdate sin pasarle argumentos, porque el objeto ya contiene la consulta.

La interfaz java.sql.PreparedStatement

```
import java.sql.Connection;
import java.sql.PreparedStatement;

// here i've already created a connection

// 1. Se crea el PreparedStatement pasando la consulta con los signos de interrogación (?)
String query = "UPDATE COFFEES SET SALES = ? WHERE COF_NAME = ?";
PreparedStatement updateSales = connection.prepareStatement(query);

// 2. Se suministran los valores para cada marcador de posición (?) usando los métodos setter.
// El primer argumento es el índice del signo de interrogación (comenzando desde el 1).
updateSales.setInt(1, 50);          // El primer '?' (SALES) será 50
updateSales.setString(2, "Espresso"); // El segundo '?' (COF_NAME) será "Espresso"

// 3. Se ejecuta la consulta llamando al método sin argumentos.
// executeUpdate devuelve un entero ('n') con la cantidad de filas que fueron actualizadas.
int n = updateSales.executeUpdate();

System.out.println("Filas actualizadas: " + n);

// Se cierra el recurso al finalizar
updateSales.close();
connection.close();
```

La interfaz java.sql.PreparedStatement

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

// 1. Here i've already created a connection. Create a PreparedStatement with a SELECT
String query = "SELECT COF_NAME, PRICE FROM COFFEES WHERE SUPPLIER_ID = ?";
PreparedStatement selectCoffees = connection.prepareStatement(query);

// 2. Supply the value for the placeholder using a setter method. This sets the first '?' to the integer value 49
selectCoffees.setInt(1, 49);

// 3. Execute the query using executeQuery(), which returns a ResultSet
// Note that no arguments are passed to executeQuery() because the statement is already prepared
ResultSet rs = selectCoffees.executeQuery();

// 4. Iterate through the ResultSet using a cursor
while (rs.next()) {
    // Retrieve column values using getter methods
    String coffeeName = rs.getString("COF_NAME");
    float price = rs.getFloat("PRICE");
    System.out.println("Coffee: " + coffeeName + ", Price: " + price);
}

rs.close();
connection.close();
```

Manejo de transacciones

Manejo de transacciones

- ¿Qué es una transacción? Es una unidad indivisible de sentencias SQL; se ejecuta todo o no se ejecuta nada
- Pasos para iniciar una transacción:
  a. Usar con.setAutoCommit(false) para desactivar el autoguardado y agrupar comandos
  a. Llamar a con.commit() para hacer los cambios permanentes en la base de datos
  a. Llamar a con.rollback() ante una SQLException o Exception de aplicación para deshacer los cambios intermedios y proteger la integridad de los datos.

Manejo de transacciones

```
try {
    // 1. Iniciar transacción (desactivar auto-guardado)
    connection.setAutoCommit(false);

    // 2. Ejecutar las sentencias SQL preparadas
    updateSales.executeUpdate();
    updateTotal.executeUpdate();

    // 3. Confirmar cambios (hacerlos permanentes)
    connection.commit();

} catch (SQLException e) {
    // 4. Deshacer todo en caso de error
    if (connection != null) con.rollback();

} finally {
    // DO NOT FORGET to close the preparedStatement objects
    // 5. Restaurar el auto-guardado al terminar
    if (connection != null) connection.setAutoCommit(true);
}
```

Buenas prácticas en el uso de JDBC

Buenas prácticas en el uso de JDBC

- **Consultas Seguras y Rápidas**: Utiliza PreparedStatement. Previene los ataques de Inyección SQL y, al estar precompilada, mejora el rendimiento si se ejecuta múltiples veces
- **Eficiencia en ResultSet**: Recupera los datos utilizando índices numéricos en lugar de nombres. Lee siempre las columnas de izquierda a derecha y una sola vez por fila
- **Transacciones**: Desactiva el auto-commit antes de empezar una transacción. Utilizar siempre el método rollback al capturar una SQLException para asegurarte de proteger la integridad de los datos si algo falla.
- **Gestión automática de recursos**: Utiliza bloques try-with-resources al instanciar tus objetos de conexión y consultas. Esto garantiza el cierre automático de los recursos (Connection, Statement, ResultSet), reduce el código repetitivo y evita fugas de memoria.

Buenas prácticas en el uso de JDBC

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
import java.sql.SQLException;

// URL de base de datos (ejemplo con sintaxis de MySQL)
String url = "jdbc:mysql://localhost:3306/midatabase";
// 1. Declaramos Connection, Statement y ResultSet dentro de los paréntesis del try.
// Se separan por punto y coma (;).
try (
    Connection con = DriverManager.getConnection(url, "usuario", "password");
    Statement stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT COF_NAME, SUPPLIER_ID FROM COFFEES")
) {

   // 2. Iteramos a través de los datos del ResultSet usando su cursor
   while (rs.next()) {
      // 3. Extraemos los valores de cada fila usando los getters
      String nombreCafe = rs.getString("COF_NAME");
      int idProveedor = rs.getInt("SUPPLIER_ID");
      System.out.println("Café: " + nombreCafe + ", Proveedor ID: " + idProveedor);
   }

} catch (SQLException e) {
   // Manejamos cualquier error que ocurra durante el proceso
   System.out.println("Ocurrió un error en la base de datos.");
   e.printStackTrace();
}

// ¡Al salir del bloque try, 'rs', 'stmt' y 'con' se cierran automáticamente!
```

Modelamiento de Herencia en BD

Modelamiento de Herencia en BD

Las entidades/clases que definamos en nuestros proyectos deberán de tener su representación respectiva en base de datos. Por ejemplo, si tengo la clase Jugador esta debería tener su respectiva tabla llamada jugador pero,

¿Qué pasaría si en mi modelo defino varios tipos de jugadores? ¿Cómo defino las tablas en BD para este escenario?

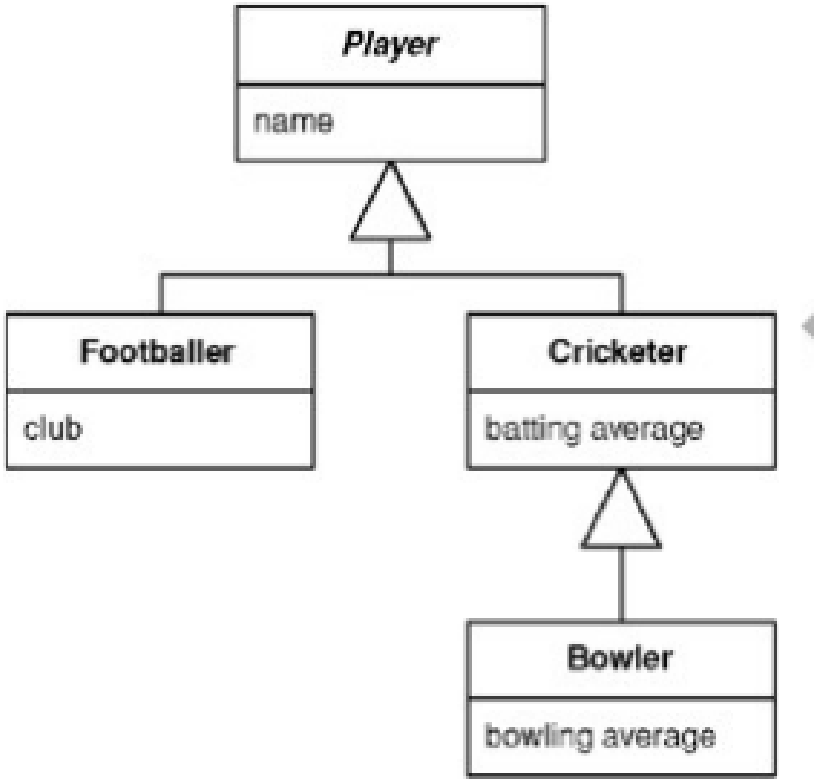


Image tomada de: https://martinfowler.com/eaaCatalog/classTableInheritance.html

Modelamiento de Herencia en BD

Single Table Inheritance: Se define una sola tabla con todos los atributos.

Para los registros donde no se tenga data en ese atributo se pondrá null.

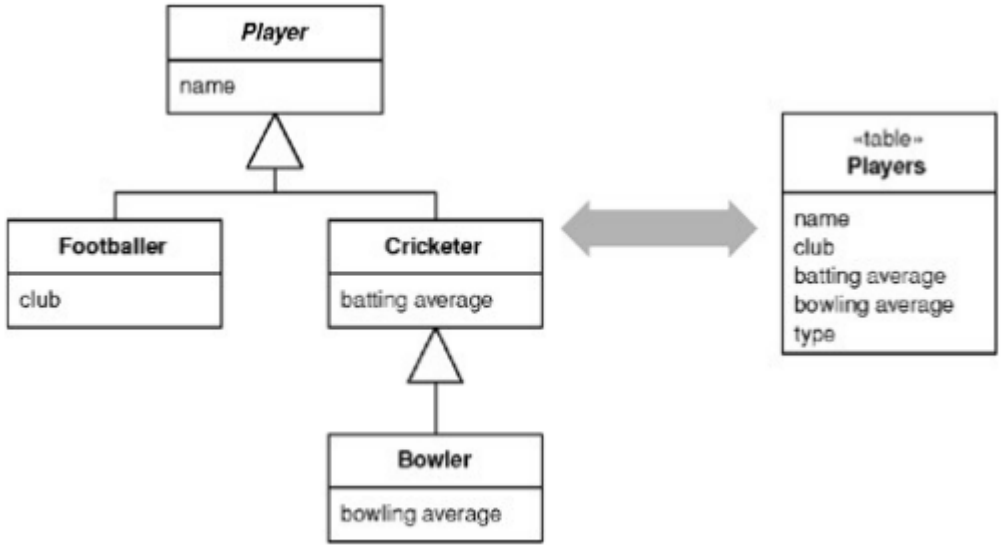


Image tomada de: https://martinfowler.com/eaaCatalog/singleTableInheritance.html

Modelamiento de Herencia en BD

Class Table Inheritance: Se define una tabla para cada clase y cada vez que se inserta un registro en la tabla hija, esta deberá registrar un registro en la tabla padre.

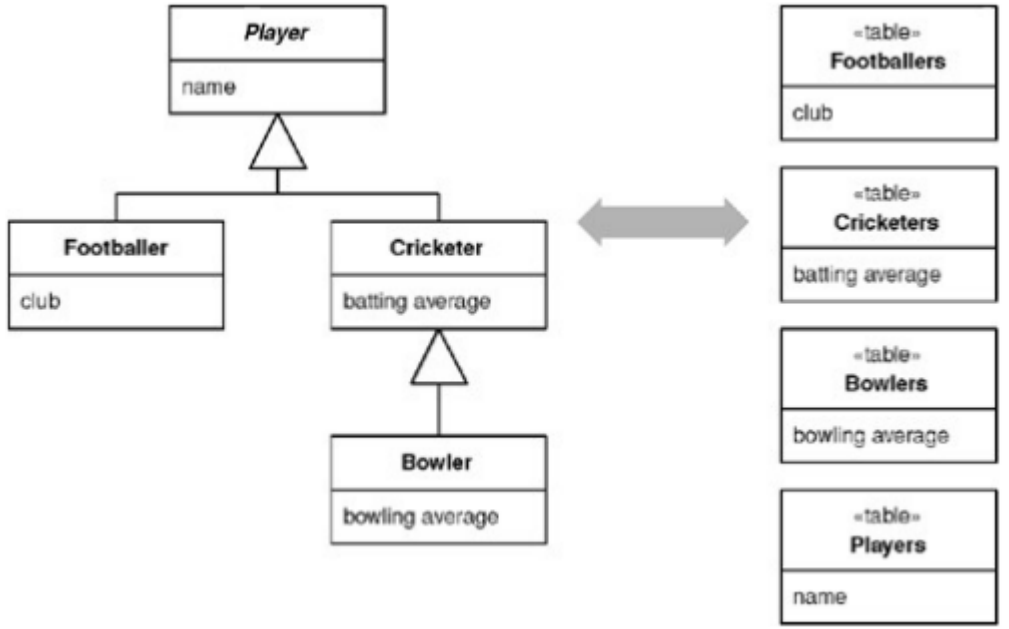


Image tomada de: https://martinfowler.com/eaaCatalog/classTableInheritance.html

Modelamiento de Herencia en BD

Concrete Table Inheritance: En este caso por cada tabla hija se crea una tabla la cual tendrá como columnas los atributos tanto de la clase hija y la clase padre (aquí la clase padre no tiene una tabla propia).

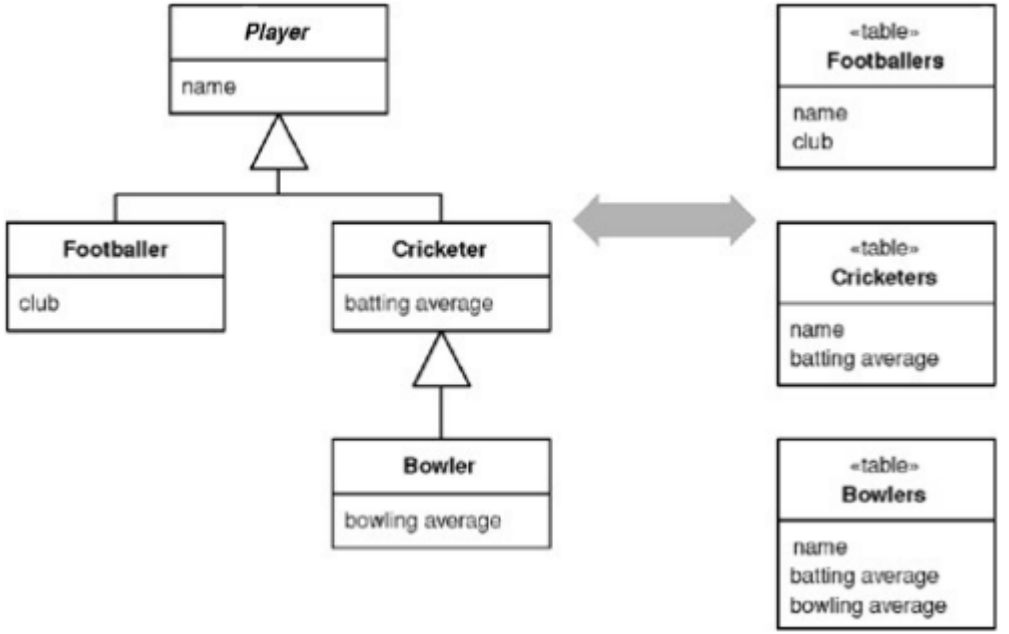


Image tomada de: https://martinfowler.com/eaaCatalog/concreteTableInheritance.html

El patrón Data Access Object (DAO)

El patrón Data Access Object (DAO)

- Es un patrón estructural que aísla la capa de negocio (tu aplicación) de la capa de persistencia (tu base de datos)
- El objetivo principal es ocultar a la aplicación toda la complejidad técnica necesaria para realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar)
- Permite cambiar la capa de persistencia de datos (por ejemplo, de una lista en memoria a MySQL) sin modificar el código de tu aplicación principal (pocos cambios)
- Se implementa utilizando una API abstracta (una Interfaz) que define las operaciones, respaldada por una clase concreta que maneja los detalles técnicos de la base de datos

El patrón Data Access Object (DAO)

- Clase de Dominio:

```
public class User {
    private Long id;
    private String name;
    private String email;

    // constructores
    // setters
    // getters estándar
}
```

- Interfaz DAO:

```
import java.util.List;

public interface UserDAO {
    User get(long id);
    List<User> getAll();
    void save(User t);
    void update(User t);
    void delete(User t);
}
```

El patrón Data Access Object (DAO)

```
import java.util.ArrayList;
import java.util.List;

public class UserDAOImpl implements UserDAO {
   private List<User> users = new ArrayList<>();

   public UserDAOImpl() {
      users.add(new User("John", "john@domain.com"));
      users.add(new User("Susan", "susan@domain.com"));
   }

   @Override
   public User get(long id) {
      return users.get(id);
   }

   @Override
   public List<User> getAll() {
      return users;
   }

   @Override
   public void save(User user) {
      users.add(user);
   }

   @Override
   public void update(User user) {
      // Logic here to update the user
   }

   @Override
   public void delete(User user) {
      users.remove(user);
   }
}
```

- Notar que la clase UserDAOImpl implementa los métodos en la interfaz UserDAO
- En este ejemplo hemos usado como repositorio de datos una lista
- Otras operaciones se puede definir en la interfaz, por ejemplo: Listar usuarios filtrados por nombre.

Definición de módulos en el proyecto SoftPro

Definición de módulos en el proyecto SoftPro

- En la clase pasada identificamos las clases que conforman el dominio o el modelo de negocio.
- Estas clases las vamos a agrugar en el módulo **softprog-model.**
- Dado que cada clase del modelo es asociada a una tabla en BD requerimos de un nivel de abstracción que nos permita realizar operaciones sobre la tabla asociada. Se define entonces el módulo **softprog-dao**.
- Finalmente, este módulo softpro-dao, requiere de conecciones a BD para realizar las diversas operaciones entonces se define el módulo **softprog-db-manager**.
- Más adelante veremos otros módulos para nuestra aplicación de ejemplo.

Definición de módulos en el proyecto SoftPro

Demo

¡Gracias!

